

QQQ: Quality Quattuor-Bit Quantization for Large Language Models

Anonymous ACL submission

Abstract

Quantization is a proven effective method for compressing large language models. Although popular techniques like W8A8 and W4A16 effectively maintain model performance, they often fail to concurrently speed up the pre-fill and decoding stages of inference. W4A8 is a promising strategy to accelerate both of them while usually leads to a significant performance degradation. To address these issues, we present QQQ, a **Quality Quattuor-bit Quantization** method with 4-bit weights and 8-bit activations. QQQ employs adaptive smoothing and Hessian-based compensation, significantly enhancing the performance of quantized models without extensive training. Furthermore, we meticulously engineer W4A8 GEMM kernels to increase inference speed. Our specialized per-channel W4A8 GEMM and per-group W4A8 GEMM achieve impressive speed increases of $3.67\times$ and $3.29\times$ over FP16 GEMM. Our extensive experiments show that QQQ achieves performance on par with existing state-of-the-art LLM quantization methods while significantly accelerating inference, achieving speed boosts up to $2.24\times$, $2.10\times$, and $1.25\times$ compared to FP16, W8A8, and W4A16, respectively.

1 Introduction

In recent years, large language models (LLMs) have been widely applied in various tasks and shown excellent performance, such as OPT (Zhang et al., 2022) and LLaMA series (Touvron et al., 2023a,b). The LLMs usually have large number of parameters and long inference time, making it inapplicable to resource-limited devices and real-time scenarios. Therefore, it is crucial to reduce LLMs’ storage and computation overhead while retaining their performance.

Quantization is a crucial technique for reducing the memory and computational requirements of LLMs. It generally involves two main strate-

gies: weight-activation and weight-only quantization. The W8A8 weight-activation quantization method, as demonstrated by SmoothQuant (Xiao et al., 2023), compresses both weights and activations to 8-bit, leveraging specialized hardware like INT8 Tensor Cores to increase computational throughput. This method excels in enhancing the compute-bound prefill phase of LLM inference. Weight-only quantization, such as the W4A16 method seen in GPTQ (Frantar et al., 2022) and AWQ (Lin et al., 2023), reduces the weight precision to 4-bit while maintaining 16-bit activations. This approach effectively cuts memory bandwidth requirements and is particularly beneficial during the memory-bound decoding phase of LLM inference. Although these methods successfully retain model performance, they struggle to simultaneously accelerate both the prefill and decoding stages of LLM inference. The W4A8 weight-activation quantization approach, which can expedite both inference phases by utilizing efficient hardware and reducing memory bandwidth, often leads to a significant decrease in model performance. Therefore, the pursuit of a viable W4A8 quantization method that balances performance with speed remains an active area of research.

In response to these challenges, we introduce QQQ, a quality W4A8 quantization approach that excels in both model performance and inference speed. QQQ employs an adaptive smoothing mechanism for activation channels with significant outliers, while leaving other channels intact. This targeted smoothing allows for more effective activation quantization. Following the smoothing process, QQQ utilizes a Hessian-based compensation technique to offset the loss from weight quantization. Through these innovative strategies, QQQ significantly improves the performance of quantized models. To achieve fast inference speed, we meticulously design specialized W4A8 GEMM kernels. These kernels are specifically tailored for

both per-channel and per-group weight quantization, drastically enhancing throughput across various batch sizes. Our contributions compared to previous works are threefold:

- We present QQQ, a quality W4A8 quantization method, that delivers performance on par with existing state-of-the-art LLM quantization methods.
- We develop innovative W4A8 GEMM kernels tailored for both per-channel weight and per-group weight quantization. These specialized kernels significantly enhance speed, with per-channel W4A8 GEMM and per-group W4A8 GEMM achieving speedups of $3.67\times$ and $3.29\times$ over FP16 GEMM, respectively.
- We integrate QQQ into vLLM (Kwon et al., 2023), achieving impressive speed boosts of $2.24\times$, $2.10\times$, and $1.25\times$ over FP16, W8A8, and W4A16 Marlin (Frantar and Alistarh, 2024) implementation, respectively.

2 Related Works

2.1 Large Language Models

In recent times, large language models (LLMs) such as GPT-3 (Brown et al., 2020), GLM (Du et al., 2021), OPT (Zhang et al., 2022), and the LLaMA series (Touvron et al., 2023a,b) have revolutionized the field of natural language processing by delivering unprecedented performance enhancements across various tasks. Despite their impressive capabilities, LLMs come with a significant computational footprint due to their billions of parameters, which takes long inference time. This makes them less suitable for deployment on devices with limited resources or in applications requiring real-time responses. For example, the GPT-3 (Brown et al., 2020) model, with its colossal count of 175 billion parameters, demands a staggering 350 GB of memory for loading its parameters in FP16 format. This necessitates the use of at least five A100-80G GPUs solely for inference purposes.

2.2 Quantization

Current strategies for quantizing LLMs fall into two main categories: weight-activation and weight-only quantization.

Weight-activation quantization. Weight-activation quantization compresses both weights and activations to lower bit representations such as

W8A8, W4A8, W6A6, and W4A4. SmoothQuant (Xiao et al., 2023), LLM.int8() (Dettmers et al.), QUIK (Ashkboos et al., 2023) and Outlier Suppression+ (Wei et al., 2023) all enhance quantized model performance by managing activation outliers. LLM.int8() (Dettmers et al.) and QUIK (Ashkboos et al., 2023) employ mixed-precision decomposition, while SmoothQuant (Xiao et al., 2023) and Outlier Suppression+ (Wei et al., 2023) use channel-wise scaling. RPTQ (Yuan et al., 2023) mitigates the impact of range differences between channels by rearranging the channels and quantizing them in clusters. OmniQuant (Shao et al., 2023) proposes learnable weight clipping and learnable equivalent transformation. Atom (Zhao et al., 2023) attains high accuracy by applying a novel mixed-precision and fine-grained quantization process. QuaRot (Ashkboos et al., 2024) removes outliers by rotating the inputs of the model using randomized Hadamard transformations.

Weight-only quantization. Weight-only quantization compresses model weights to lower bit precision (e.g., 4-bit or 2-bit) while keeping activations at higher precision (16-bit). GPTQ (Frantar et al., 2022) adopts second-order information to minimize precision loss. SpQR (Dettmers et al., 2023) and AWQ (Lin et al., 2023) prioritize the preservation of important weights to reduce quantization errors, with SpQR applying mixed-precision and AWQ optimizing per-channel scaling. QuIP (Chee et al., 2024) further quantizes weights to 2-bit based on the insight that model parameters should ideally be incoherent.

3 Methodology

In this section, we detail our quantization method, referred to as QQQ. As depicted in Figure 1, QQQ is a two-stage weight-activation quantization method. It integrates adaptive smoothing and Hessian-based quantization compensation to enhance the performance of quantized models. QQQ adaptively smooths activation channels with significant outliers while leaving other channels intact, enabling better activation quantization. After smoothing the activations, QQQ tackles the challenges of weight quantization by employing Hessian-based compensation, which effectively minimizes the loss incurred during the quantization process. To expedite practical inference, QQQ incorporates highly efficient W4A8 GEMMs tailored for both per-channel and per-group quantization.

The quantization process will be further elaborated in the following subsections.

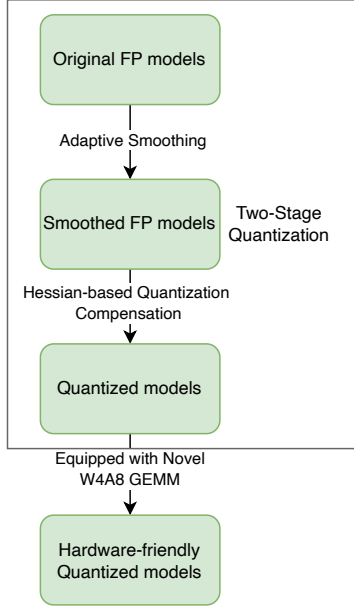


Figure 1: Framework of QQQ.

3.1 Adaptive Smoothing

We employ channel-wise smoothing to refine problematic activation channels with outliers, enhancing their suitability for quantization. As indicated by research (Wei et al., 2023), smoothing only the activation channels with significant outliers while leaving other channels intact can enhance the performance of quantized models. Therefore, we further implement an adaptive smoothing strategy that identifies and selectively smooths only those channels that display significant outliers.

Specifically, we introduce an outlier threshold σ , which is determined through a grid search within the range $[0, \max(\text{abs}(\mathbf{X}))]$. Activation channels that exhibit outliers exceeding this threshold σ are then selectively smoothed:

$$\mathcal{T} = \{t \mid \max(\text{abs}(\mathbf{X}_{:,t})) < \sigma\}, \quad (1)$$

where $\mathcal{T}$ denotes the selected channel index for $\mathbf{X}$.

To optimize the smoothing parameters, we aim to reduce the squared error between the product of activations and weights both before and after quantization:

$$\arg \min_{\mathbf{s}} \|Q(\mathbf{X}_{:, \mathcal{T}} \oslash \mathbf{s})Q(\mathbf{W}_{\mathcal{T},:} \odot \mathbf{s}) - \mathbf{X}_{:, \mathcal{T}} \mathbf{W}_{\mathcal{T},:}\|_2^2, \quad (2)$$

where Q denotes the quantization function; $\mathbf{W}$ denotes the unquantized weight matrix; $\mathbf{X}$ denotes

the unquantized activation matrix; $\mathbf{s}$ denotes the smoothing parameter; $\oslash$ and $\odot$ are element-wise division and multiplication, respectively.

3.2 Hessian-based Quantization Compensation

After transferring the quantization challenge from activations to weights through adaptive smoothing, it becomes crucial to enhance the quantization technique for weights. We adopt a layer-wise quantization framework GPTQ (Frantar et al., 2022), which advocates for a precise and efficient Hessian-based quantization compensation technique. This method involves an application of two key formulas, Eq.(3) and Eq.(4), which are instrumental in the iterative quantization of weights. Specifically, this algorithm adjusts the subset F of full-precision weights by an increment δ_F , compensating for the quantization error induced by the quantized weights $Q(\mathbf{W}_i)$.

$$\mathbf{W}_i = \underset{\mathbf{W}_i}{\operatorname{argmin}} \frac{(Q(\mathbf{W}_i) - \mathbf{W}_i)^2}{[\mathbf{H}_F^{-1}]_{ii}} \quad (3)$$

$$\delta_F = -\frac{\mathbf{W}_i - Q(\mathbf{W}_i)}{[\mathbf{H}_F^{-1}]_{ii}} \cdot (\mathbf{H}_F^{-1})_{:,i} \quad (4)$$

where $\mathbf{W}_i$ denotes the i -row of weight matrix $\mathbf{W}$, $\mathbf{H}_F = 2\mathbf{X}_F^T \mathbf{X}_F$ denotes the Hessian matrix.

3.3 Novel W4A8 GEMM

We carefully develop innovative W4A8 GEMMs tailored to optimize the inference speed. It's worth noting that our design exclusively supports symmetric quantization due to its superior hardware efficiency compared to asymmetric quantization.

The W4A8 GEMM design introduces two critical considerations beyond conventional GEMM implementations. Firstly, as GPUs support GEMM operations only for identical data types, it's necessary to convert INT4 weights into INT8 to utilize the capabilities of the INT8 Tensor Cores. Secondly, to avoid potential overflow during INT8 GEMM, the final output is typically stored in INT32 format, which subsequently requires a dequantization setup from INT32 to FP16.

To further reduce memory access, we integrate both the type conversion and dequantization process within the W4A8 GEMM routine. We have different W4A8 GEMM configurations for per-channel weight quantization and per-group weight quantization, as they have different quantization schemes, which will be detailed in the following subsections.

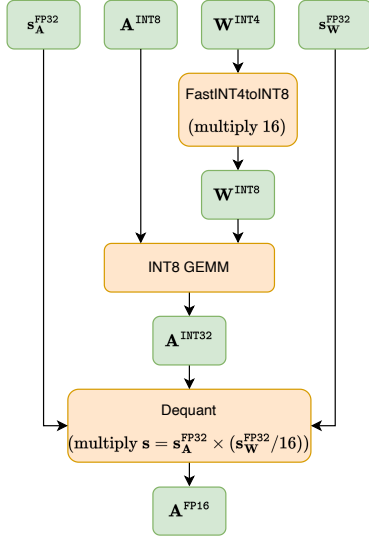


Figure 2: The W4A8 GEMM for per-channel weight quantization. $\mathbf{A}$ denotes the activation matrix. $\mathbf{W}$ denotes the weight matrix. $s_{\mathbf{A}}$ denotes the per-token quantization scale of activations. $s_{\mathbf{W}}$ denotes the per-channel quantization scale of weights. The superscript represents the data type. The green sections indicate **data**, and the orange sections indicate **operations**.

3.3.1 Per-channel Weight Quantization

The specialized W4A8 GEMM process for per-channel weight quantization is depicted in Figure 2. Our method involves an initial conversion of INT4 weights to INT8 before executing the INT8 GEMM, followed by a dequantization step that transforms the INT32 output of the INT8 GEMM into a FP16 result. To ensure high efficiency in the type conversion, we utilize the FastINT4toINT8 (Li et al., 2023) technique. This process involves positioning the INT4 weight into the upper 4 bits of an INT8 by multiplying by 16, essentially performing a left shift by 4 bits. Once the GEMM computation is complete, we scale down the result by a factor of 16 to obtain the accurate output. This scaling operation is seamlessly integrated into the dequantization step, Dequant, by adjusting the weight quantization scale offline by the same factor.

3.3.2 Per-group Weight Quantization.

The W4A8 GEMM tailored for per-group weight quantization is detailed in Figure 3. This approach involves a more complex set of operations prior to the INT8 GEMM compared to the per-channel weight quantization. Although the conversion from INT4 to INT8 weights is still necessary, additional steps are required due to the unique per-group quantization scales that are not directly compatible with standard GEMM procedures.

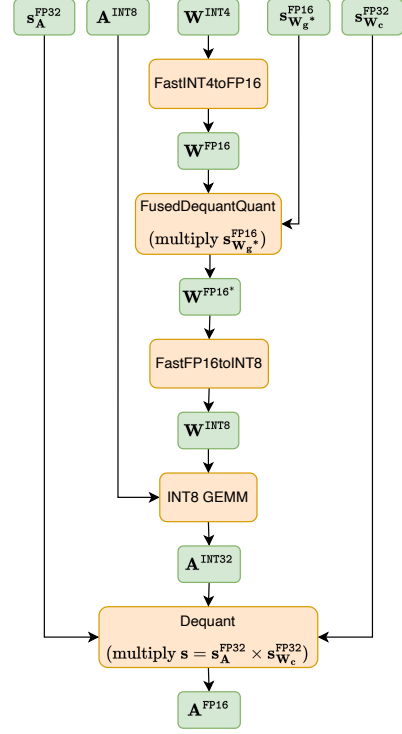


Figure 3: The W4A8 GEMM for per-group weight quantization. $s_{\mathbf{W}_c}$ represents the per-channel quantization scale converting from dequantized FP16 weights back to INT8 weights. $s_{\mathbf{W}_g}^*$ represents the fused per-group scale converting from INT4 weights to INT8 weights.

Initially, INT4 weights are converted to FP16 using the FastINT4toFP16 (Kim et al., 2022) method. Subsequently, each group’s weight is dequantized by multiplying it by its respective scale. To re-quantize into INT8, the result is divided by the per-channel quantization scale. These two steps are elegantly merged into a single operation, FusedDequantQuant, by dividing the per-group scale by the per-channel scale, as illustrated in Figure 4. The final stage of the process involves converting the FP16 results of FusedDequantQuant back into INT8 format to proceed with the INT8 GEMM. Similar to the per-channel method, the INT32 output from the INT8 GEMM undergoes dequantization to FP16.

The conventional instruction for converting FP16 to INT8 is significantly slow, which hinders the overall GEMM performance. To tackle this issue, we introduce a fast conversion technique, FastFP16toINT8, as depicted in Figure 5. The conversion begins by adjusting the FP16 values, adding 128 to shift them into the range of $[0.0, 255.0]$, which corresponds to the representational span of UINT8. This is followed by an addition of 1024, effectively aligning the 8 bits of

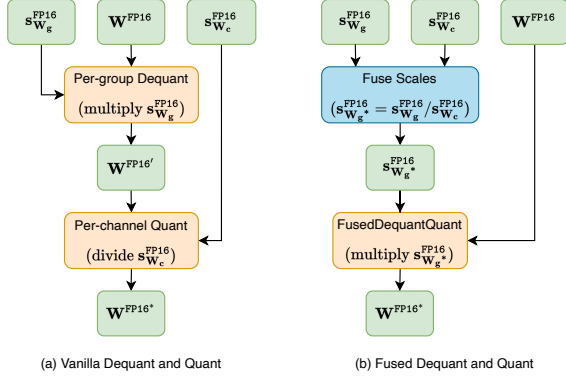


Figure 4: Design of FusedDequantQuant. s_{W_g} represents the per-group quantization scale converting from FP16 weights to INT4 weights. The orange sections indicate **online operations**, and the blue sections indicate **offline operations**.

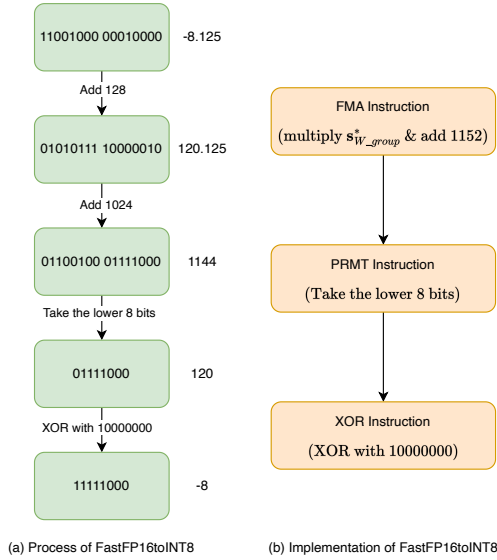


Figure 5: Design of FastFP16toINT8.

UINT8 within the lower segment of FP16’s mantissa. The final step involves extracting these lower 8 bits from FP16 and applying an XOR operation with 0x80 to achieve the desired INT8 format. In practice, the fusion of the per-group quantization scale multiplication with the addition of the magic number $128 + 1024$ is executed through a single FMA (Fused Multiply-Add) instruction. Furthermore, the process leverages two expedient bit manipulation instructions, PRMT and XOR, to finalize the conversion.

4 Experiments

4.1 Setups

In this section, we introduce the experimental configurations in models, datasets, algorithm and inference system.

Models and Datasets. We conduct the experiments using the LLaMA series (Touvron et al., 2023a,b). We evaluate the model performance on language modeling and zero-shot tasks. For the language modeling aspect, we utilize the WikiText2 (Merity et al., 2016) dataset to measure perplexity. For the zero-shot tasks, we extend the evaluation to a variety of datasets, including PIQA (Bisk et al., 2020), HellaSwag (Zellers et al., 2019), WinoGrande (Sakaguchi et al., 2021), and ARC (Clark et al., 2018), utilizing the lm_eval (Gao et al., 2023) toolkit for a comprehensive analysis. Additionally, we select a subset of 128 sequences from the PILE (Gao et al., 2020) dataset to facilitate calibration.

Algorithm. QQQ is specifically designed to focus on per-token quantization for activations and supports both per-channel and per-group quantization for weights. To optimize inference speed, QQQ exclusively uses symmetric quantization for both activations and weights, thus avoiding the additional computational overhead associated with asymmetric quantization.

Inference System. The inference system that we have develop to support QQQ is based on vLLM¹ (Kwon et al., 2023) v0.4.1. The system’s capabilities are further enhanced by the implementation of W4A8 GEMM kernels, which are based on the advanced Marlin² (Frantar and Alistarh, 2024) kernels. All experimental procedures are conducted on NVIDIA A100 80G GPUs under PyTorch 2.1.2 with CUDA 11.8.

4.2 Model Performance

We perform a comprehensive comparison of QQQ method against leading LLM quantization methods, including SmoothQuant (Xiao et al., 2023), GPTQ (Frantar et al., 2022), AWQ (Lin et al., 2023), Atom (Zhao et al., 2023), and the recently introduced W4A8 method QoQ (Lin et al., 2024). By default, weights employ per-channel quantization. GPTQ, AWQ, and QoQ utilize asymmetric quantization for weights, whereas SmoothQuant, Atom, and QQQ opt for symmetric quantization. In terms of activation quantization, Atom stands out by implementing per-group quantization, while the remaining methods, including QQQ, employ per-token quantization. Across all methods under consideration, activations apply symmetric quantization.

¹<https://github.com/vllm-project/vllm>

²<https://github.com/IST-DASLab/marlin>

#Bits	Method	LLaMA-1			LLaMA-2			LLaMA-3
		7B	13B	30B	7B	13B	70B	8B
FP16	-	5.68	5.09	4.10	5.47	4.88	3.32	6.14
W8A8	SmoothQuant	5.78	5.17	4.28	5.58	4.93	3.39	6.25
W4A16	GPTQ-g128	5.83	5.20	4.22	5.63	4.99	3.43	6.56
	AWQ-g128	5.78	5.19	4.21	5.60	4.97	3.41	6.54
W4A4	Atom-g128	6.25	5.52	4.61	6.12	5.31	3.73	7.76
W4A8	QoQ	5.93	5.28	4.34	5.75	5.12	3.52	6.89
	QoQ-g128	5.89	5.25	4.28	5.70	5.08	3.47	6.76
	QQQ	6.19	5.43	4.61	5.95	5.21	3.68	7.41
	QQQ-g128	5.87	5.24	4.30	5.71	5.01	3.50	6.64

Table 1: WikiText2 perplexity with 2048 sequence length. The lower is the better. ‘-g128’ denotes applying per-group quantization on the weights and the group size is 128.

LLaMA-2	#Bits	Method	PIQA	ARC-e	ARC-c	HellaSwag	WinoGrande	Avg.
7B	FP16	-	79.05	74.58	46.25	76.05	68.98	68.98
	W4A4	Atom-g128	75.14	52.99	38.40	69.37	62.75	59.73
	W4A8	QoQ	77.64	72.81	43.60	74.00	68.03	67.22
		QoQ-g128	78.07	73.32	44.80	74.98	68.59	67.95
		QQQ	77.42	69.15	42.15	73.54	65.98	65.65
		QQQ-g128	78.51	72.94	44.37	74.53	67.01	67.47
13B	FP16	-	80.52	77.44	49.06	79.38	72.22	71.72
	W4A4	Atom-g128	76.50	57.49	42.32	73.84	67.40	63.51
	W4A8	QoQ	79.71	75.97	48.38	77.80	70.96	70.56
		QoQ-g128	79.43	77.06	48.81	78.35	70.48	70.83
		QQQ	79.43	74.75	48.12	77.27	70.32	69.98
		QQQ-g128	79.98	76.64	48.55	78.63	71.82	71.13

Table 2: Zero-shot accuracy on five common sense tasks with 2048 sequence length. The higher is the better.

Table 1 presents a detailed perplexity comparison on the WikiText2 dataset. The per-group QQQ demonstrates competitive performance with W8A8 SmoothQuant, W4A16 GPTQ, and W4A16 AWQ across various models. For example, per-group QQQ only increases perplexity by up to 0.13 on LLaMA-2-13B compared with them. When benchmarked against W4A4 Atom, both the per-channel and per-group QQQ consistently outperform Atom on all tested models. For instance, per-group QQQ achieves a significant reduction in perplexity by 0.41 on LLaMA-2-13B when compared to Atom. In comparison with W4A8 QoQ, per-group QQQ outperforms per-group QoQ on the majority of models, exemplified by a 0.12 decrease in perplexity on LLaMA-3-8B. However, it is observed that per-channel QQQ falls slightly behind

per-channel QoQ. This divergence is hypothesized to be due to QoQ’s application of asymmetric quantization on weights, as opposed to the symmetric quantization employed by QQQ.

The zero-shot accuracy evaluation across five common sense tasks is showed in Table 2. The QQQ method exhibits a significant performance edge over Atom across all tasks and model sizes. For instance, the per-group QQQ achieves a 4.79% higher accuracy than Atom on the HellaSwag task when utilizing the LLaMA-2-13B model. In comparison with QoQ, QQQ demonstrates comparable results, especially when per-group quantization is applied. For instance, QQQ shows a marginal improvement of 0.3% over QoQ with the LLaMA-2-13B model on average.

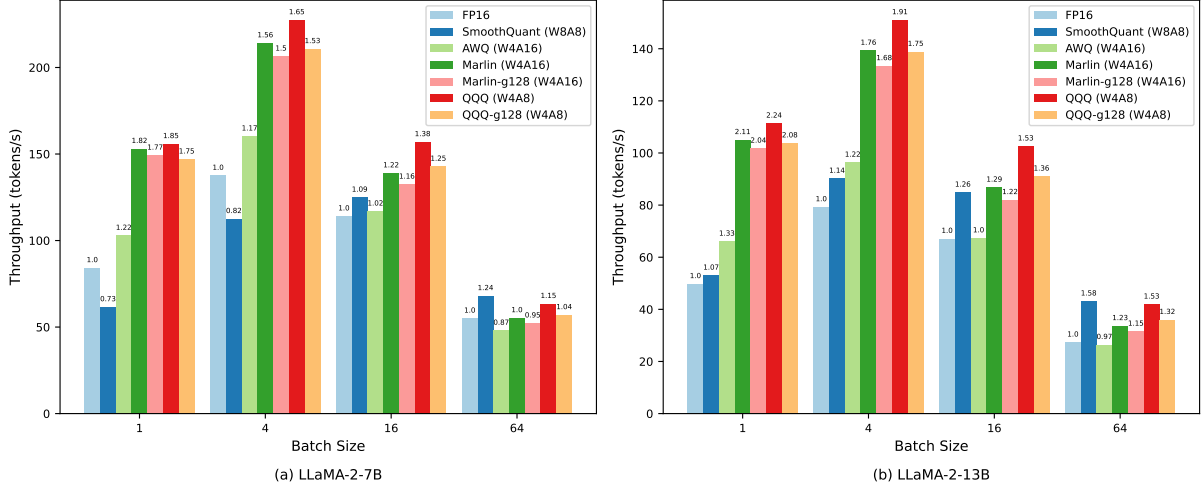


Figure 6: Same-batch throughput comparison of quantized LLaMA-2 models under various batch size.

4.3 Speedup

In the evaluation of inference efficiency, QQQ is benchmarked against a suite of quantization methods including SmoothQuant³ (Xiao et al., 2023), AWQ (Lin et al., 2023), and GPTQ (Frantar et al., 2022), all of which are integrated within the vLLM (Kwon et al., 2023). For optimal inference speed, we leverage the Marlin (Frantar and Alistarh, 2024) implementation of GPTQ. All methods are tested under conditions that enable continuous batching and paged attention. The benchmarks are conducted with an input sequence length of 1024 and an output sequence length of 128.

The throughput comparison outlined in Figure 6 indicates that QQQ outperforms the alternative methods across a range of batch sizes in most cases. Specifically, QQQ attains a substantial speed increase, reaching up to $2.24\times$, $2.10\times$, $1.59\times$, and $1.25\times$ faster than FP16, SmoothQuant, AWQ, and Marlin on the LLaMA-2-13B model, respectively. When contrasted with W4A16 AWQ and W4A16 Marlin, QQQ displays a comparable inference speed at smaller batch sizes but gains a significant advantage as the batch size increases. Against W8A8 SmoothQuant, QQQ stands out with an approximate $2.0\times$ speedup for smaller batches and maintains a similar level of acceleration even at a batch size of 64. Moreover, QQQ’s performance enhancement is more pronounced on larger models, as evidenced by the greater speedup achieved on LLaMA-2-13B compared to LLaMA-2-7B.

³<https://github.com/vllm-project/vllm/pull/1508>

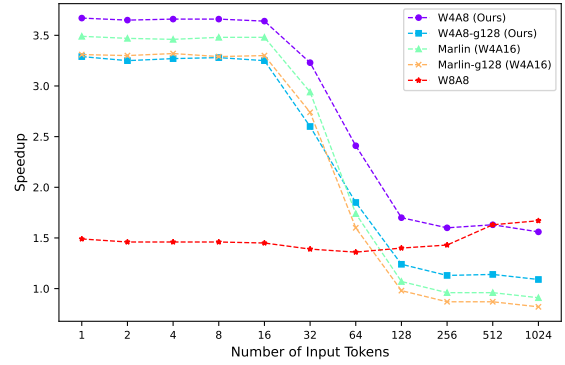


Figure 7: Speedup over PyTorch FP16 GEMM (Calling CUTLASS) of all GEMMs under different numbers of input tokens. The weight matrix size is ($N = 8192$, $K = 21760$).

4.4 W4A8 GEMM Studies

We conduct a series of experiments to demonstrate the capabilities of our W4A8 GEMMs. By keeping the weight matrix size constant and varying the number of input tokens, we evaluate the speed improvements over the PyTorch FP16 GEMM benchmark. The results are depicted in Figure 7.

Our W4A8 GEMMs demonstrate impressive speedups, achieving up to $3.67\times$, $2.51\times$, and $1.71\times$ faster performance compared to PyTorch FP16 GEMM, W8A8 GEMM, and W4A16 Marlin GEMM, respectively. Specifically, W4A8 GEMMs show a significant advantage when the token count is below 64 compared to W8A8 GEMM. With an increasing number of tokens, the per-group W4A8 GEMM maintains a similar level of efficiency, whereas the per-channel W4A8 GEMM tends to be slower. This discrepancy is attributed to the higher type conversion overhead in per-group

Granularity	Model	B	B+AS	B+AS+GPTQ
Per-channel	LLaMA-1-7B	6.93	6.77	6.19
	LLaMA-1-13B	5.84	5.84	5.43
	LLaMA-1-30B	5.04	4.89	4.61
	LLaMA-2-7B	7.36	7.29	5.95
	LLaMA-2-13B	5.47	5.43	5.21
	LLaMA-2-70B	4.07	3.88	3.68
	LLaMA-3-8B	10.88	9.46	7.41
Per-group	LLaMA-1-7B	6.16	5.96	5.87
	LLaMA-1-13B	5.43	5.32	5.24
	LLaMA-1-30B	4.51	4.34	4.30
	LLaMA-2-7B	5.86	5.77	5.71
	LLaMA-2-13B	5.10	5.06	5.01
	LLaMA-2-70B	3.67	3.52	3.50
	LLaMA-3-8B	7.11	6.96	6.64

Table 3: WikiText2 perplexity of LLaMA series when applying various quantization techniques. 'B' represents the Baseline, which refers to the vanilla W4A8 method without any additional quantization techniques. 'AS' denotes Adaptive Smoothing.

W4A8 GEMM. Compared to Marlin GEMM, our GEMMs demonstrate superior performance under the majority of token counts, especially per-group GEMM. As the token count rises, Marlin GEMM's performance gradually declines, falling behind even the FP16 GEMM, while our GEMMs consistently outperform the FP16 benchmark. This clearly indicates the superior efficiency of INT8 Tensor Cores over FP16 Tensor Cores in compute-bound scenarios.

4.5 Ablation Studies

4.5.1 Impact of Quantization Techniques

Here we examine the impact of different quantization techniques on model performance. We use the standard W4A8 method, which lacks any specialized quantization techniques, as the baseline for comparison. The perplexity results for the LLaMA series are summarized in Table 3. Overall, the combination of adaptive smoothing with GPTQ proves to be the most effective method, yielding the lowest perplexity scores. For instance, this approach reduces perplexity by 0.47 in per-group quantization compared with the baseline.

4.5.2 Impact of FastFP16toINT8

Here we explore the performance enhancements achieved in per-group W4A8 GEMM through the implementation of the FastFP16toINT8 conversion technique. We benchmark the optimized

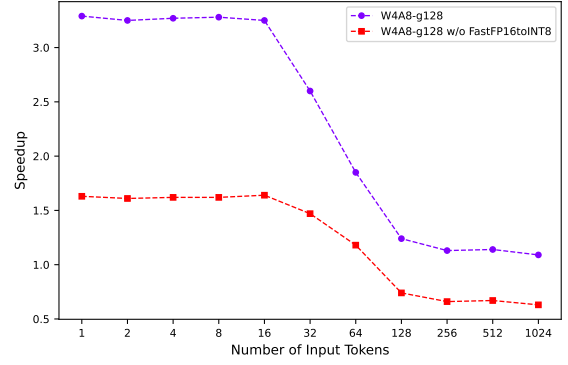


Figure 8: Speedup over PyTorch FP16 GEMM (Calling CUTLASS) of two per-group W4A8 GEMMs under different numbers of input tokens. The weight matrix size is ($N = 8192, K = 21760$).

W4A8 GEMM against the conventional W4A8 GEMM, which utilizes the standard FP16 to INT8 conversion instruction. As illustrated in Figure 8, the GEMM utilizing FastFP16toINT8 demonstrates a significant increase in speedup compared to its counterpart without this optimization, across a range of input tokens. The FastFP16toINT8 technique can accelerate per-group W4A8 GEMM by up to $2.02\times$. This finding confirms that the FP16 to INT8 conversion process is a critical bottleneck, and our FastFP16toINT8 effectively enhances the efficiency of per-group W4A8 GEMM by mitigating this constraint.

5 Conclusion

In this paper, we propose a quality W4A8 quantization method optimized for hardware efficiency. QQQ leverages the power of adaptive smoothing combined with Hessian-based quantization compensation to significantly improve the performance of quantized models. In addition, we meticulously develop W4A8 GEMM kernels to maximize the inference speed. Through extensive experiments, QQQ demonstrates superior performance over existing state-of-the-art LLM quantization methods in terms of both performance and speed. We believe QQQ can serve as a powerful tool that opens up access to the deployment and application of LLMs.

Limitations

QQQ method has two limitations: 1) The two-stage quantization method requires more time for quantizing models. 2) The mixed-precision GEMMs only support 4-bit weights for now.

References

- Saleh Ashkboos, Iliia Markov, Elias Frantar, Tingxuan Zhong, Xincheng Wang, Jie Ren, Torsten Hoefler, and Dan Alistarh. 2023. Towards end-to-end 4-bit inference on generative large language models. *arXiv preprint arXiv:2310.09259*.
- Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L Croci, Bo Li, Martin Jaggi, Dan Alistarh, Torsten Hoefler, and James Hensman. 2024. Quarot: Outlier-free 4-bit inference in rotated llms. *arXiv preprint arXiv:2404.00456*.
- Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. 2020. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.
- Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher M De Sa. 2024. Quip: 2-bit quantization of large language models with guarantees. *Advances in Neural Information Processing Systems*, 36.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*.
- Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. Llm.int8(): 8-bit matrix multiplication for transformers at scale, 2022. *CoRR abs/2208.07339*.
- Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefler, and Dan Alistarh. 2023. Spqr: A sparse-quantized representation for near-lossless llm weight compression. *arXiv preprint arXiv:2306.03078*.
- Zhengxiao Du, Yujie Qian, Xiao Liu, Ming Ding, Jiezhong Qiu, Zhilin Yang, and Jie Tang. 2021. Glm: General language model pretraining with autoregressive blank infilling. *arXiv preprint arXiv:2103.10360*.
- Elias Frantar and Dan Alistarh. 2024. Marlin: a fast 4-bit inference kernel for medium batchsizes. <https://github.com/IST-DASLab/marlin>.
- Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*.
- Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. 2020. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*.
- Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2023. A framework for few-shot language model evaluation.
- Young Jin Kim, Rawn Henry, Raffy Fahim, and Hany Hassan Awadalla. 2022. Who says elephants can’t run: Bringing large scale moe models into cloud scale production. *arXiv preprint arXiv:2211.10017*.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- Qingyuan Li, Ran Meng, Yiduo Li, Bo Zhang, Liang Li, Yifan Lu, Xiangxiang Chu, Yerui Sun, and Yuchen Xie. 2023. A speed odyssey for deployable quantization of llms. *arXiv preprint arXiv:2311.09550*.
- Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. 2023. Awq: Activation-aware weight quantization for llm compression and acceleration. *arXiv preprint arXiv:2306.00978*.
- Yujun Lin, Haotian Tang, Shang Yang, Zhekai Zhang, Guangxuan Xiao, Chuang Gan, and Song Han. 2024. Qserve: W4a8kv4 quantization and system co-design for efficient llm serving. *arXiv preprint arXiv:2405.04532*.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*.
- Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavata, and Yejin Choi. 2021. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106.
- Wenqi Shao, Mengzhao Chen, Zhaoyang Zhang, Peng Xu, Lirui Zhao, Zhiqian Li, Kaipeng Zhang, Peng Gao, Yu Qiao, and Ping Luo. 2023. Omniquant: Omnidirectionally calibrated quantization for large language models. *arXiv preprint arXiv:2308.13137*.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023a. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023b. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Xiuying Wei, Yunchen Zhang, Yuhang Li, Xiangguo Zhang, Ruihao Gong, Jinyang Guo, and Xianglong Liu. 2023. Outlier suppression+: Accurate quantization of large language models by equivalent and optimal shifting and scaling. *arXiv preprint arXiv:2304.09145*.
- Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023. Smoothquant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning*, pages 38087–38099. PMLR.
- Zhihang Yuan, Lin Niu, Jiawei Liu, Wenyu Liu, Xingang Wang, Yuzhang Shang, Guangyu Sun, Qiang Wu, Jiaxiang Wu, and Bingzhe Wu. 2023. Rptq: Reorder-based post-training quantization for large language models. *arXiv preprint arXiv:2304.01089*.
- Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*.
- Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. 2022. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*.
- Yilong Zhao, Chien-Yu Lin, Kan Zhu, Zihao Ye, Lequn Chen, Size Zheng, Luis Ceze, Arvind Krishnamurthy, Tianqi Chen, and Baris Kasikci. 2023. Atom: Low-bit quantization for efficient and accurate llm serving. *arXiv preprint arXiv:2310.19102*.